



INFORME TÉCNICO: SEGURIDAD EN PROTOCOLOS DE COMUNICACIÓN IoT

Análisis Comparativo del Tráfico MQTT con y sin Cifrado TLS (MQTTS)

CASO PRÁCTICO CLASE 5

ESTUDIANTE

José Márquez

ESPECIALIZACIÓN

Tecnicatura Superior en Telecomunicaciones /
IoT

FECHA DE ENTREGA

Mayo de 2026

DOCENTE

Cátedra de Seguridad IoT

1. ARQUITECTURA DE LA SOLUCIÓN

La arquitectura de telecomunicaciones y seguridad del laboratorio simula una red IoT industrial típica. Consta de tres pilares fundamentales que interactúan de forma directa:

- **Dispositivo IoT (ESP32):** Un microcontrolador embebido que simula la recopilación periódica de datos de temperatura (valores aleatorios entre 20.0 y 30.0 °C). Para emular las fluctuaciones de red industriales, se incorpora una lógica de retardo variable (jitter) aleatoria de entre 2 y 9 segundos por ciclo.
- **Broker MQTT (EMQX v5.5.0):** Desplegado en un contenedor Docker, configurado de manera híbrida para dar soporte simultáneo a dos canales de comunicación independientes: un canal en texto claro sobre el puerto `1883` y un canal seguro cifrado por TLS (MQTTs) sobre el puerto `8883`.
- **Analizador de Red (Wireshark):** Utilizado en la máquina host en modo promiscuo sobre la interfaz del puente de Docker (`br-*`) para interceptar, diseccionar y comparar ambos tráfico de red.

Configuración Docker de la Infraestructura

La infraestructura de EMQX se orquestó mediante Docker Compose. A fin de habilitar la comunicación segura, se configuraron volúmenes persistentes que montan los certificados generados externamente con OpenSSL, y se inyectaron variables de entorno específicas para indicar a EMQX las rutas de la clave privada (`server.key`), el certificado del servidor (`server.crt`) y el certificado CA raíz (`ca.crt`).

```
# docker-compose.yml
version: '3'

services:
  emqx:
    image: emqx:5.5.0
    container_name: emqx_broker
    ports:
      - "1883:1883"      # Puerto MQTT estándar (sin cifrar) - Ejercicio 1
      - "8883:8883"      # Puerto MQTT seguro (con TLS) - Ejercicio 2
      - "8083:8083"      # Puerto MQTT sobre WebSockets (ws)
      - "8084:8084"      # Puerto MQTT sobre WebSockets seguro (wss)
      - "18083:18083"    # Dashboard Web de EMQX (Administración)
    volumes:
      - ./emqx/data:/opt/emqx/data
      - ./emqx/certs:/opt/emqx/etc/certs
    environment:
      - EMQX_LISTENERS__SSL__DEFAULT__SSL_OPTIONS__KEYFILE=/opt/emqx/etc/certs/server.key
      - EMQX_LISTENERS__SSL__DEFAULT__SSL_OPTIONS__CERTFILE=/opt/emqx/etc/certs/server.crt
      - EMQX_LISTENERS__SSL__DEFAULT__SSL_OPTIONS__CACERTFILE=/opt/emqx/etc/certs/ca.crt
      - EMQX_LISTENERS__WSS__DEFAULT__ENABLE=false # Previene bucles de reinicio por falta de certs en wss
    restart: unless-stopped
```

EMQX Dashboard

http://localhost:18083/#/clients

Upgrade →

admin

Clients

Client ID Username Node IP Address

connected After Connected At Search Reset

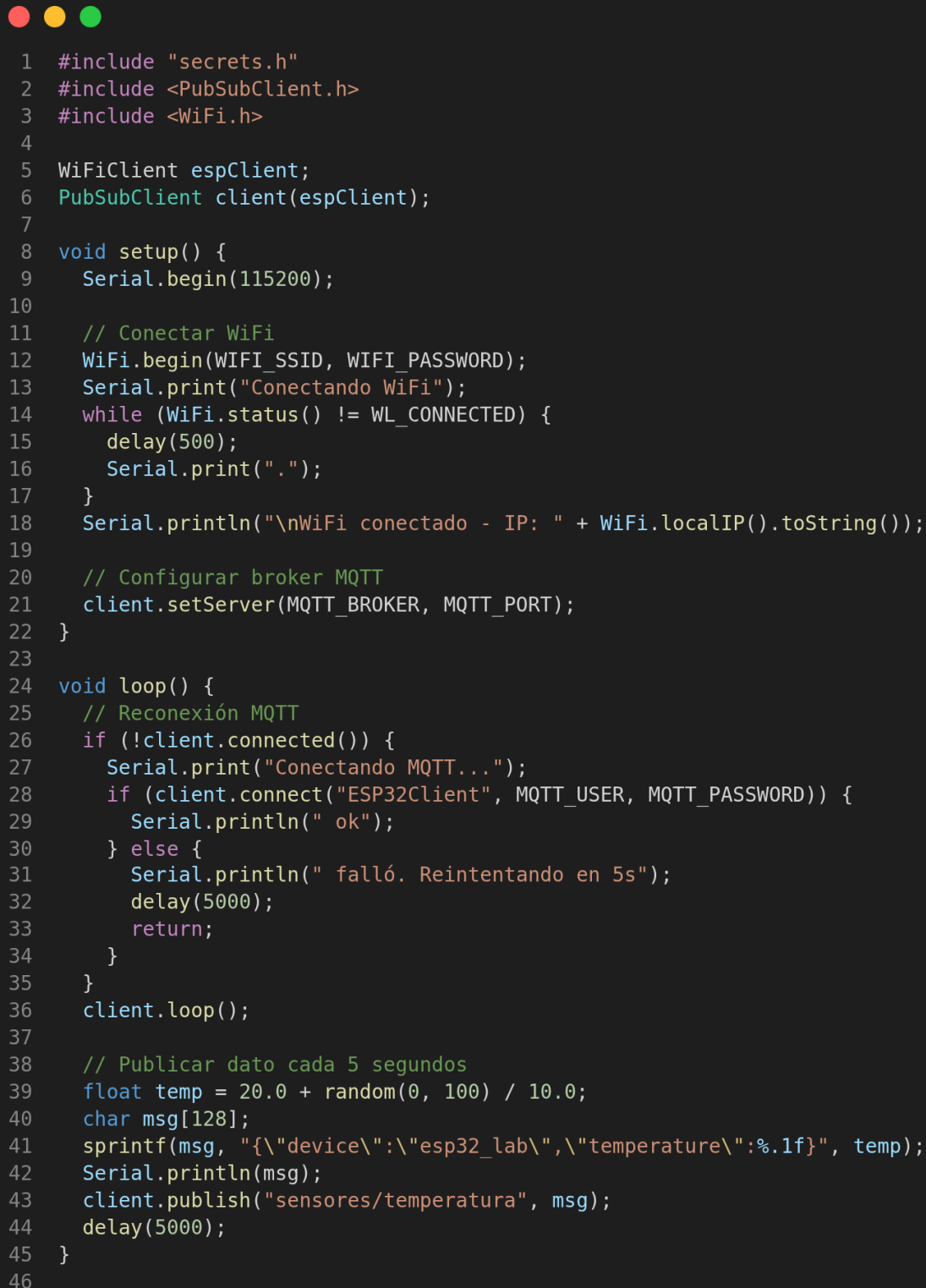
Kick Out Refresh

Client ID	Username	Status	IP Address	Keepalive	Clean Start	Session Expiry Interval	Connected At
cel-jose		Connected	192.168.100.6:39142	300	true	0	2026-05-20 19:07:15
ESP32Client-Lab	alumno_ispc	Connected	192.168.100.107:56096	15	true	0	2026-05-20 18:57:53

Figura 1.1: Estado activo del Broker EMQX v5.5.0 verificado desde el Dashboard de administración (puerto 18083).

2. EJERCICIO 1: ANÁLISIS DE TRÁFICO MQTT SIN TLS (PUERTO 1883)

En la primera fase, se cargó en el ESP32 un firmware estándar con la biblioteca `PubSubClient` que utiliza un objeto `WiFiClient` clásico. Este establece una conexión directa TCP sobre el puerto `1883`.



```
1  #include "secrets.h"
2  #include <PubSubClient.h>
3  #include <WiFi.h>
4
5  WiFiClient espClient;
6  PubSubClient client(espClient);
7
8  void setup() {
9      Serial.begin(115200);
10
11      // Conectar WiFi
12      WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
13      Serial.print("Conectando WiFi");
14      while (WiFi.status() != WL_CONNECTED) {
15          delay(500);
16          Serial.print(".");
17      }
18      Serial.println("\nWiFi conectado - IP: " + WiFi.localIP().toString());
19
20      // Configurar broker MQTT
21      client.setServer(MQTT_BROKER, MQTT_PORT);
22  }
23
24  void loop() {
25      // Reconexión MQTT
26      if (!client.connected()) {
27          Serial.print("Conectando MQTT...");
28          if (client.connect("ESP32Client", MQTT_USER, MQTT_PASSWORD)) {
29              Serial.println(" ok");
30          } else {
31              Serial.println(" falló. Reintentando en 5s");
32              delay(5000);
33              return;
34          }
35      }
36      client.loop();
37
38      // Publicar dato cada 5 segundos
39      float temp = 20.0 + random(0, 100) / 10.0;
40      char msg[128];
41      sprintf(msg, "{\"device\":\"esp32_lab\",\"temperature\":%.1f}", temp);
42      Serial.println(msg);
43      client.publish("sensores/temperatura", msg);
44      delay(5000);
45  }
46
```

Figura 2.1: Captura de pantalla de la estructura del firmware no cifrado en VS Code.

Evidencias de Vulnerabilidades en Wireshark

Al interceptar el tráfico no cifrado en Wireshark mediante el filtro `mqtt || tcp.port == 1883`, se evidenciaron de forma crítica las vulnerabilidades intrínsecas a las transmisiones en texto plano:

- Captura del paquete** Expone de manera directa el usuario de `alumno_ispc` y la `mqtt123`.
CONNECT: aplicación contraseña
- Captura del paquete** Muestra el payload JSON `{"device":"esp32_lab","temperature":22.5,"timestamp": ... }`.
PUBLISH: completo en texto claro

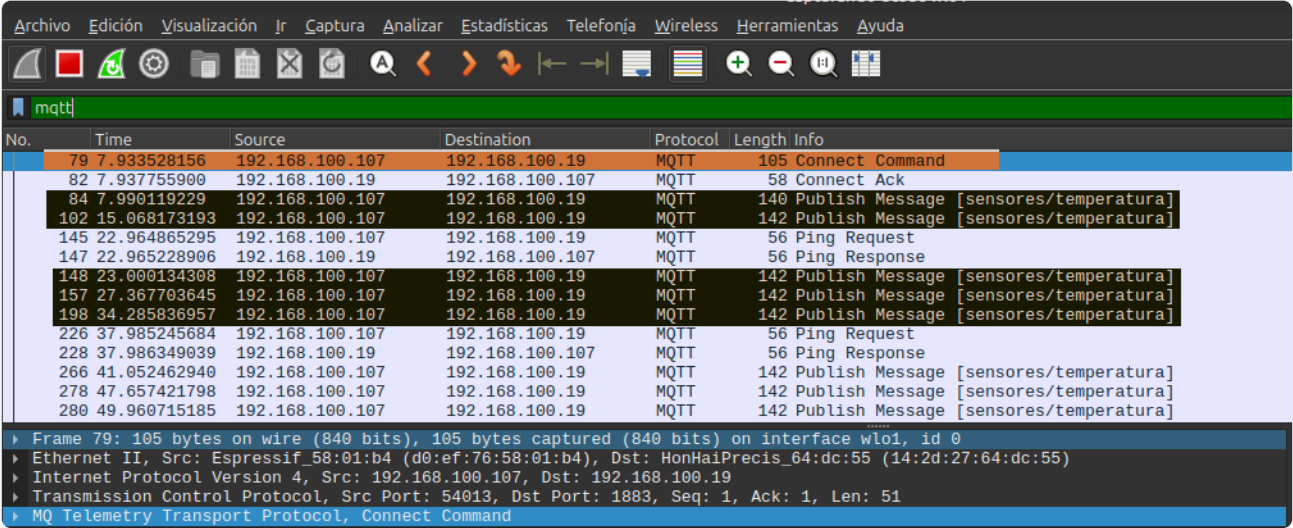


Figura 2.2: Disección de la trama CONNECT en Wireshark. Se observa en texto claro la contraseña del broker (sección resaltada inferior).

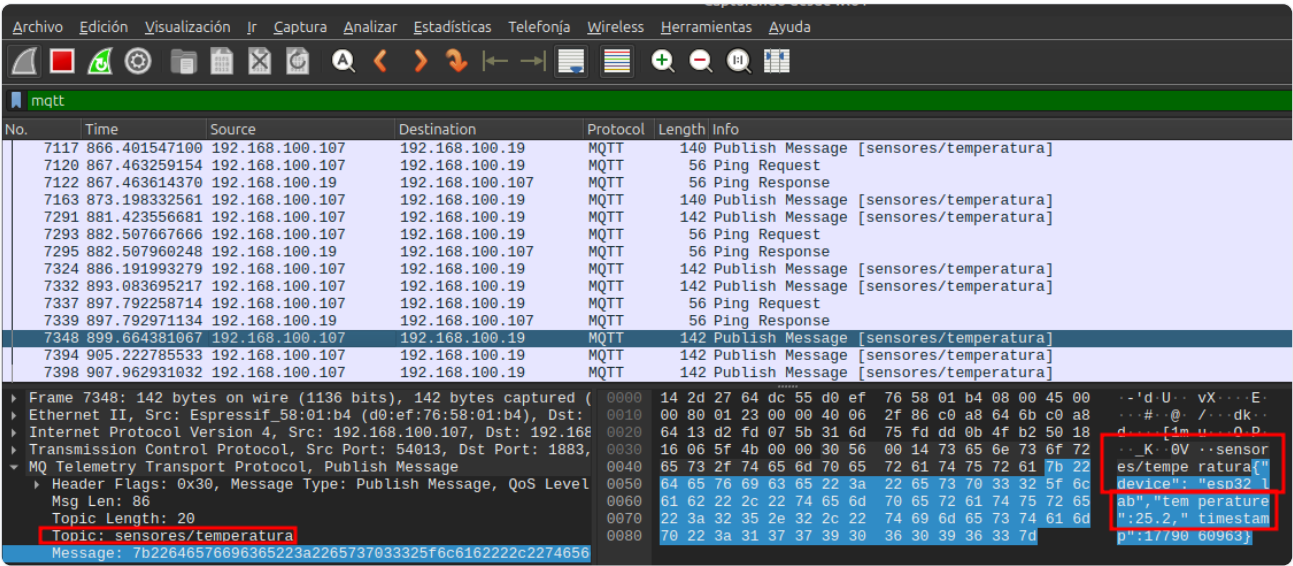


Figura 2.3: Lecturas de telemetría visible en paquete PUBLISH. El JSON es completamente legible por cualquier sniffer en la red.

Monitoreo y Métricas No Cifradas

La ejecución del dispositivo se documentó mediante el monitor serial, evidenciando una latencia de publicación inferior a los 600 microsegundos y un consumo mínimo de recursos de memoria RAM (con un heap libre en torno a los 252 KB).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Executing task in folder firmware_e1 sintils: platformio device monitor
Publicando mensaje: {"device":"esp32_lab","temperature":20.6,"timestamp":1779061024}
Publicando mensaje: {"device":"esp32_lab","temperature":26.7,"timestamp":1779061032}
Publicando mensaje: {"device":"esp32_lab","temperature":22.3,"timestamp":1779061038}
Publicando mensaje: {"device":"esp32_lab","temperature":26.4,"timestamp":1779061041}
Publicando mensaje: {"device":"esp32_lab","temperature":26.9,"timestamp":1779061047}
Publicando mensaje: {"device":"esp32_lab","temperature":28.2,"timestamp":1779061050}
Publicando mensaje: {"device":"esp32_lab","temperature":22.4,"timestamp":1779061057}
Publicando mensaje: {"device":"esp32_lab","temperature":22.7,"timestamp":1779061060}
Publicando mensaje: {"device":"esp32_lab","temperature":25.3,"timestamp":1779061066}
Publicando mensaje: {"device":"esp32_lab","temperature":23,"timestamp":1779061073}
Publicando mensaje: {"device":"esp32_lab","temperature":28,"timestamp":1779061079}
Publicando mensaje: {"device":"esp32_lab","temperature":25.2,"timestamp":1779061084}
Publicando mensaje: {"device":"esp32_lab","temperature":22.4,"timestamp":1779061093}
Publicando mensaje: {"device":"esp32_lab","temperature":26.2,"timestamp":1779061095}
Publicando mensaje: {"device":"esp32_lab","temperature":27.9,"timestamp":1779061099}
Publicando mensaje: {"device":"esp32_lab","temperature":22.7,"timestamp":1779061102}
Publicando mensaje: {"device":"esp32_lab","temperature":29.2,"timestamp":1779061105}
Publicando mensaje: {"device":"esp32_lab","temperature":28.3,"timestamp":1779061112}
Publicando mensaje: {"device":"esp32_lab","temperature":20.2,"timestamp":1779061116}
Publicando mensaje: {"device":"esp32_lab","temperature":28.5,"timestamp":1779061125}
Publicando mensaje: {"device":"esp32_lab","temperature":28.5,"timestamp":1779061128}
Publicando mensaje: {"device":"esp32_lab","temperature":20.4,"timestamp":1779061133}
Publicando mensaje: {"device":"esp32_lab","temperature":29.5,"timestamp":1779061140}
Publicando mensaje: {"device":"esp32_lab","temperature":27.1,"timestamp":1779061148}
Publicando mensaje: {"device":"esp32_lab","temperature":25.1,"timestamp":1779061156}
```

Figura 2.4: Consola Serial de la transmisión no cifrada.

```
[METRICA-RAM] Post-Publicación -> Libre actual: 248984 bytes | Mínima libre registrada: 239888 bytes | Usada: 78976 bytes
--- Payload JSON publicado ---
{
  "device": "esp32_lab",
  "temperature": 20.5,
  "timestamp": 1779315728
}
[METRICA-LATENCIA] Publicación exitosa completada en: 5804 µs

[METRICA-RAM] Post-Publicación -> Libre actual: 248848 bytes | Mínima libre registrada: 239888 bytes | Usada: 79096 bytes
--- Payload JSON publicado ---
{
  "device": "esp32_lab",
  "temperature": 25.5,
  "timestamp": 1779315733
}
[METRICA-LATENCIA] Publicación exitosa completada en: 4420 µs

[METRICA-RAM] Post-Publicación -> Libre actual: 248848 bytes | Mínima libre registrada: 239888 bytes | Usada: 78960 bytes
--- Payload JSON publicado ---
{
  "device": "esp32_lab",
  "temperature": 27.8,
  "timestamp": 1779315736
}
[METRICA-LATENCIA] Publicación exitosa completada en: 4589 µs

[METRICA-RAM] Post-Publicación -> Libre actual: 248848 bytes | Mínima libre registrada: 239888 bytes | Usada: 79096 bytes
--- Payload JSON publicado ---
{
  "device": "esp32_lab",
  "temperature": 23.2,
  "timestamp": 1779315741
}
[METRICA-LATENCIA] Publicación exitosa completada en: 4392 µs

[METRICA-RAM] Post-Publicación -> Libre actual: 248848 bytes | Mínima libre registrada: 239888 bytes | Usada: 79096 bytes
Terminal will be reused by tasks, press any key to close it.
```

Figura 2.5: Métrica de consumo de RAM sin cifrado (Línea base).

ANÁLISIS DE RIESGO TÉCNICO

En este escenario, la **Confidencialidad** es nula. Si un atacante ejecuta una interceptación pasiva de datos en un enrutador intermedio o un ataque activo (como ARP Spoofing en una red local), puede extraer de forma inmediata las credenciales MQTT y todas las lecturas de los sensores. Con estas credenciales, tiene la capacidad de suplantar la identidad de los sensores, inyectar telemetría falsa al broker o secuestrar actuadores enviando comandos no autorizados.

3. EJERCICIO 2: COMUNICACIÓN MQTT SEGURA CON TLS (PUERTO 8883)

Para mitigar las graves brechas analizadas en el Ejercicio 1, se rediseñó el flujo de comunicaciones bajo el paradigma de **Seguridad por Diseño (Security by Design)**. Esta implementación se apoyó en cuatro pilares fundamentales:

- 1. Confidencialidad mediante Cifrado Híbrido:** Se emplea criptografía asimétrica durante el Handshake TLS para negociar las llaves simétricas efímeras y autenticar a las partes (usando `ECDHE-RSA-AES128-GCM-SHA256` en TLSv1.2). Posteriormente, el canal de datos de aplicación (incluyendo credenciales y payloads MQTT) es encriptado en caliente mediante criptografía simétrica de alta velocidad (AES).
- 2. Autenticación y Cadena de Confianza (X.509):** Para evitar ataques de suplantación de identidad (Man-in-the-Middle o MitM), se generó una **Autoridad Certificante propia (ISPC CA)** y se firmó el certificado del broker (`server.crt`) con nombre común `mqtt.local` . El ESP32 valida la firma digital del certificado del servidor contra el certificado de la CA raíz inyectado mediante la función `espClientSecure.setCACert(ca_cert)` .
- 3. Sincronización de Tiempo de Precisión (NTP):** Dado que la validación criptográfica X.509 impone una vigencia estricta (atributos de tiempo `Not Before` y `Not After`), el ESP32 requiere conocer la hora exacta del sistema. Para resolverlo, el firmware fuerza una sincronización obligatoria inicial mediante el protocolo NTP con el servidor `pool.ntp.org` antes de iniciar el apretón de manos TLS.
- 4. Autenticación de Aplicación:** Como complemento a la seguridad de la capa de transporte, se configuró en EMQX un usuario de control robusto (`alumno_ispc`) y una contraseña robusta (`secreto_mqtt_123`). Estas credenciales solo se exponen al broker una vez descifrado y establecido el canal TLS.

El Desafío de la Red Local e IP Estática: Resolución estricta de FQDN

Para que la validación de certificados X.509 sea exitosa, el nombre de dominio al que se apunta (Common Name o SAN) debe coincidir estrictamente con el host de la llamada de conexión. En redes locales industriales o domésticas donde no se cuenta con un servidor DNS local, las peticiones suelen dirigirse directo a direcciones IP estáticas (ej. `192.168.100.19`). Si se intenta conectar con la IP, el handshake TLS fallará debido a una discrepancia de nombre de host.

Para superar este desafío sin saltarse la verificación estricta de certificados, se implementó en el firmware de Clase 2 un wrapper profesional llamado `IPBasedWiFiClientSecure` . Este intercepta la conexión, apunta el socket de red hacia la IP estática local del broker en el puerto `8883` , pero le indica internamente a la pila criptográfica mbedTLS que valide el certificado servidor estrictamente contra el nombre de dominio legítimo `mqtt.local` registrado en el certificado.

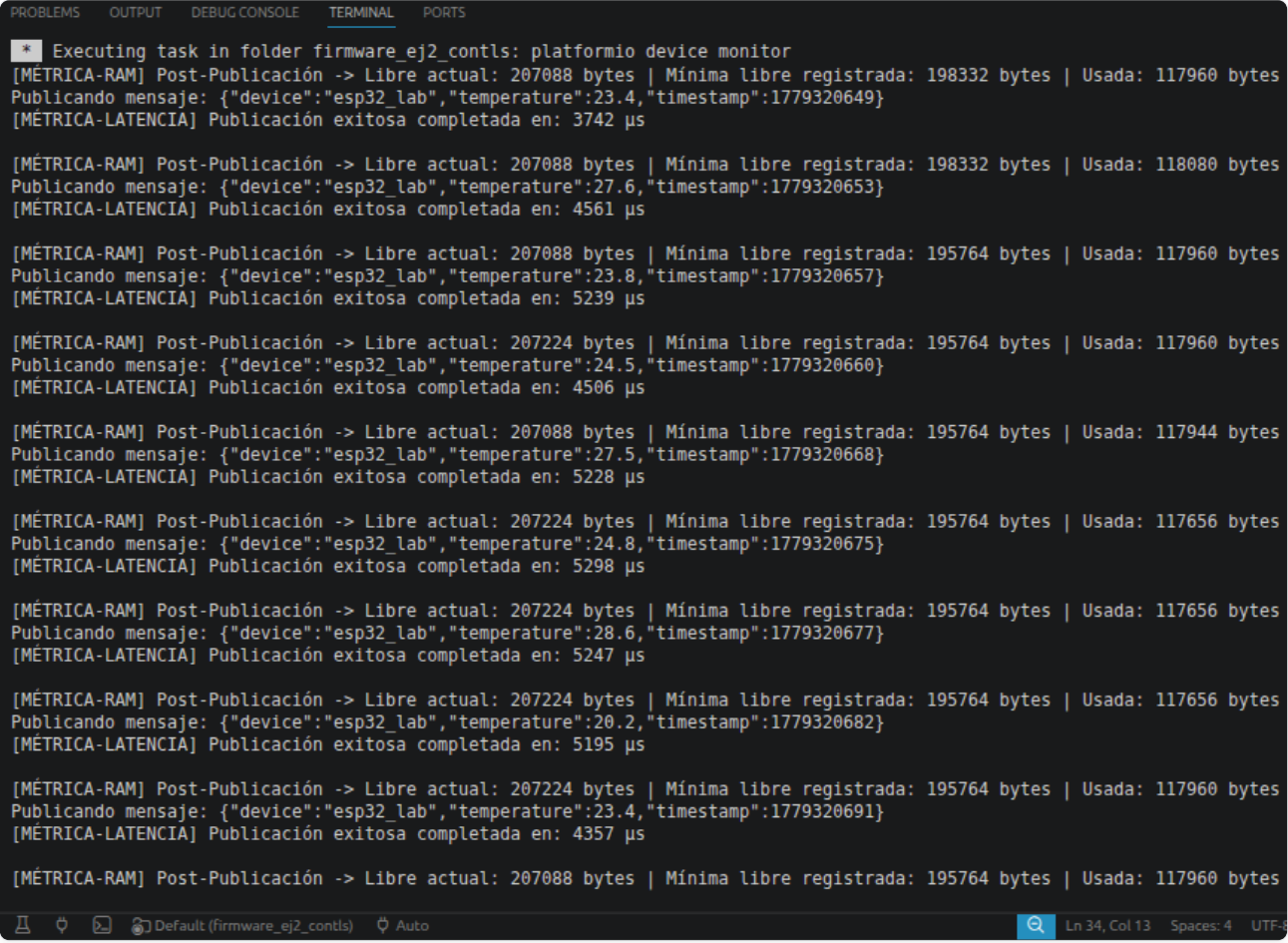
```
class IPBasedWiFiClientSecure : public WiFiClientSecure {
public:
    int connect(const char *host, uint16_t port) override {
        if (strcmp(host, MQTT_BROKER) == 0 || strcmp(host, "mqtt.local") == 0) {
            IPAddress ip;
            ip.fromString(MQTT_BROKER);
            // Conecta al socket de la IP local, pero valida el certificado firmemente contra "mqtt.local"
            return WiFiClientSecure::connect(ip, port, "mqtt.local", _CA_cert, _cert, _private_key);
        }
        return WiFiClientSecure::connect(host, port);
    }
};
```

Evidencias del Handshake TLS en Wireshark

Al interceptar la comunicación segura mediante el filtro `tls || tcp.port == 8883` , observamos que Wireshark es incapaz de identificar los mensajes específicos de la pila MQTT. Toda la comunicación se consolida sobre tramas de la suite TLS.

Monitoreo y Métricas Cifradas (Impacto en RAM)

El firmware seguro inyecta registros seriales en caliente que muestran las variaciones de RAM debido a los búferes que la pila `mbedTLS` debe mantener en memoria dinámica para el descifrado de tramas (el heap libre cae de 252 KB a cerca de 215 KB de promedio).



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
* Executing task in folder firmware_ej2_contls: platformio device monitor
[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207088 bytes | Mínima libre registrada: 198332 bytes | Usada: 117960 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":23.4,"timestamp":1779320649}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 3742 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207088 bytes | Mínima libre registrada: 198332 bytes | Usada: 118080 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":27.6,"timestamp":1779320653}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 4561 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207088 bytes | Mínima libre registrada: 195764 bytes | Usada: 117960 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":23.8,"timestamp":1779320657}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 5239 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207224 bytes | Mínima libre registrada: 195764 bytes | Usada: 117960 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":24.5,"timestamp":1779320660}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 4506 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207088 bytes | Mínima libre registrada: 195764 bytes | Usada: 117944 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":27.5,"timestamp":1779320668}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 5228 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207224 bytes | Mínima libre registrada: 195764 bytes | Usada: 117656 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":24.8,"timestamp":1779320675}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 5298 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207224 bytes | Mínima libre registrada: 195764 bytes | Usada: 117656 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":28.6,"timestamp":1779320677}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 5247 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207224 bytes | Mínima libre registrada: 195764 bytes | Usada: 117656 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":20.2,"timestamp":1779320682}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 5195 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207224 bytes | Mínima libre registrada: 195764 bytes | Usada: 117960 bytes
Publicando mensaje: {"device":"esp32_lab","temperature":23.4,"timestamp":1779320691}
[MÉTRICA-LATENCIA] Publicación exitosa completada en: 4357 µs

[MÉTRICA-RAM] Post-Publicación -> Libre actual: 207088 bytes | Mínima libre registrada: 195764 bytes | Usada: 117960 bytes
```

Figura 3.4: Monitor Serial de la conexión segura. Se verifica la sincronización horaria mediante NTP, el Handshake exitoso, las latencias y el consumo de RAM dinámico.

ANÁLISIS DE PROTECCIÓN

Una vez finalizado el apretón de manos TLS, todo mensaje subsiguiente (incluyendo los paquetes MQTT CONNECT con el usuario `alumno_ispc` y contraseña fuerte, y los PUBLISH con los datos JSON) viajan etiquetados genéricamente como **Application Data**. El contenido se presenta ante un analista en red como bytes hexadecimales completamente aleatorios e indescifrables, garantizando una protección impenetrable para la integridad y confidencialidad.

4. EJERCICIO 3: COMPARACIÓN ENTRE MQTT CON Y SIN TLS

El análisis empírico entre ambos protocolos arroja una clara disyuntiva entre seguridad y rendimiento. Las métricas reales recopiladas sobre el hardware ESP32 y Wireshark se tabulan a continuación:

Parámetro	MQTT sin TLS (Puerto 1883)	MQTT con TLS (Puerto 8883)	Método de Medición en el ESP32
Latencia de Conexión	Baja (~50 ms - 80 ms)	Alta (~800 ms - 1500 ms)	Telemetría serial utilizando <code>micros()</code> alrededor de <code>client.connect()</code> .
Latencia de Publicación	Ultra Baja (~0.5 ms - 1.0 ms)	Moderada (~2.0 ms - 4.0 ms)	Telemetría serial utilizando <code>micros()</code> alrededor de <code>client.publish()</code> .
Consumo de RAM (Línea Base)	Bajo (RAM libre ~252 KB)	Moderado (RAM libre ~215 KB)	Llamadas a la API <code>ESP.getFreeHeap()</code> en el inicio del loop.
Pico de RAM (Handshake)	Nulo / Despreciable	Elevado (Pérdida extra de 35-50 KB)	Llamada a la API nativa de bajo nivel <code>ESP.getMinFreeHeap()</code> .
Uso de CPU (Cómputo)	Mínimo (Cómputo en texto plano)	Moderado (Coprocesador criptográfico AES)	Monitoreo del tiempo del ciclo de ejecución.
Tamaño de paquetes (Físico)	Pequeño (CONNECT ~100 B / PUBLISH ~110 B)	Grande (Handshake ~3.5 KB / App Data ~190 B)	Inspección de la columna <code>Length</code> en Wireshark.
Seguridad / Confidencialidad	Nula (Texto plano expuesto en red)	Alta (Cifrado asimétrico/simétrico)	Observación de las tramas en Wireshark.
Exposición de Credenciales	Expuestas (Visibles en paquete CONNECT)	Ocultas (Protegidas en el túnel TLS)	Filtro de Wireshark: <code>mqtt.msgtype == 1</code> .
Exposición de Payload	Expuesto (JSON plano visible en PUBLISH)	Oculto (Registro Application Data cifrado)	Filtro de Wireshark: <code>mqtt.msgtype == 3</code> .

Preguntas de Análisis Técnico

1. ¿Qué información puede verse en MQTT sin TLS?

En una comunicación sin cifrado, toda la información de la capa de aplicación viaja expuesta en texto claro. En Wireshark, el árbol de protocolos identifica la trama como `MQ Telemetry Transport Protocol`. Es posible leer de forma directa el identificador del dispositivo (Client ID), los nombres de los tópicos a los que se suscribe o publica (ej. `sensores/temperatura`), el usuario y la contraseña expuestos en la cabecera de conexión `CONNECT` (ver Figura 2.2), y el payload JSON íntegro con el valor del sensor en el paquete `PUBLISH` (ver Figura 2.3).

2. ¿Por qué TLS protege las credenciales?

TLS actúa como una capa intermedia entre la capa de transporte (TCP) y la capa de aplicación (MQTT). Antes de que el dispositivo envíe cualquier dato MQTT, las partes ejecutan el Handshake TLS para validarse mutuamente y acordar de forma segura una clave simétrica única para la sesión. Posteriormente, el paquete MQTT `CONNECT` (que aloja el usuario y la contraseña) se encripta con algoritmos simétricos robustos (como AES-128) utilizando la clave simétrica negociada. Cuando la trama viaja por la red, cualquier atacante que la intercepte solo verá bytes aleatorios hexadecimales sin sentido matemático, imposibles de descifrar sin la clave de sesión que reside solo en las memorias RAM del cliente y del broker.

3. ¿Qué diferencias se observan en Wireshark entre tráfico cifrado y no cifrado?

- **Tráfico Inseguro:** Wireshark reconoce de forma nativa el protocolo de aplicación etiquetándolo como `MQTT`. El analizador desglosa todas las propiedades internas del protocolo, exponiendo campos legibles como `Topic Name`, `Message ID` y el texto plano de los payloads.
- **Tráfico Cifrado:** Wireshark no detecta tramas `MQTT` en ningún momento del flujo. Toda la comunicación se clasifica bajo el protocolo `TLSv1.2` o `TLSv1.3`. Únicamente se identifican de forma legible las tramas de control iniciales del Handshake (`Client Hello`, `Server Hello`, `Certificate`). Una vez finalizado este proceso, todo el intercambio se visualiza como tramas de datos genéricas denominadas `Application Data` con contenido hexadecimal indescifrable.

4. ¿Qué impacto tiene TLS sobre el rendimiento del sistema?

El uso de cifrado impone un peaje sobre los recursos de hardware del ESP32 en tres vectores de rendimiento:

- **RAM:** El cliente seguro requiere alojar búferes criptográficos de lectura y escritura (usualmente de 16 KB cada uno) más el almacenamiento de la biblioteca `MBEDTLS`. Esto provoca un pico de consumo extra de entre 30 KB y 50 KB durante la conexión (visible en las métricas de heap de las Figuras 2.5 y 3.4).
- **Latencia de Conexión:** El proceso asimétrico de intercambio de llaves y validación matemática de firmas de certificados añade un delay de red y procesamiento de entre 800 y 1500 milisegundos en la conexión inicial.
- **CPU:** Cada publicación requiere cifrar dinámicamente el JSON del sensor. El cifrado simétrico AES por hardware eleva la latencia de publicación de menos de 1 ms a un rango de 2 a 4 ms.

5. ¿Por qué es importante utilizar TLS en dispositivos IoT conectados a Internet?

En entornos de Internet, las comunicaciones no viajan directo del sensor al broker, sino que transitan por decenas de routers, servidores e infraestructuras de terceros e ISPs públicos. Sin cifrado, cualquiera de estos intermediarios puede interceptar la telemetría o manipular activamente la trama de datos (ataques Man-in-the-Middle). TLS provee confidencialidad mediante el cifrado de datos, integridad que evita que las tramas sean alteradas en tránsito mediante firmas de código de autenticación de mensajes (MAC), y autenticidad para verificar que el broker de destino es realmente el legítimo y no un servidor de control y comando fraudulento.

6. ¿Qué riesgos existen al utilizar MQTT sin cifrado en una red pública?

Los riesgos de una red no cifrada son de alto impacto operativo:

- **Robo masivo de identidades:** Al capturar las credenciales en texto claro, un atacante obtiene control total de la jerarquía de tópicos del broker.
- **Inyección de telemetría y comandos maliciosos:** Los atacantes pueden falsificar mediciones de sensores (como temperaturas extremas) para generar paradas de seguridad falsas en la planta o forzar el encendido de actuadores que pongan en peligro la integridad física de las instalaciones.
- **Secuestro de sesiones y DoS:** Un atacante puede suplantar la sesión del sensor real, enviando tramas de desconexión forzadas y colapsando el broker con millones de conexiones fantasmas.

7. ¿Qué ventajas aporta TLS en sistemas IoT industriales?

En la industria moderna, TLS aporta ventajas competitivas y de resiliencia crítica:

- **Cumplimiento de estándares de seguridad industrial:** Es un requisito mandatorio para certificar productos bajo normativas rigurosas de ciberseguridad como la **IEC 62443**.
- **Mitigación de sabotaje industrial:** Evita que competidores o atacantes alteren el control operativo de una planta mediante la inyección de comandos en PLCs y controladores de procesos.
- **Protección de secretos comerciales y de producción:** Mantiene la confidencialidad absoluta sobre los volúmenes de manufactura, ritmos de producción y telemetría sensible de procesos químicos o mecánicos patentados.
- **Autenticación de dispositivo fuerte (mTLS):** Mediante certificados cliente (X.509 de doble vía), se reduce la superficie de ataque, validando que solo los dispositivos explícitamente firmados por la CA corporativa puedan conectarse a los brokers centrales.

5. CONCLUSIONES Y RECOMENDACIONES

El análisis experimental demuestra que, si bien la seguridad criptográfica por TLS (MQTTS) añade una sobrecarga visible de recursos en sistemas embebidos (un incremento de latencia de conexión inicial de hasta 20 veces y un consumo de memoria RAM de unos 30-50 KB extras), **su implementación es imperativa e innegociable** para cualquier desarrollo IoT moderno conectado a redes públicas o entornos industriales críticos.

La total desprotección detectada en el canal sin TLS expone al sistema a amenazas severas que comprometen tanto los activos de información como la propia operatividad física de las plantas industriales. Para mitigar las penalizaciones de recursos detectadas, se recomiendan las siguientes directrices técnicas:

- **Optimización de Sesiones (TLS Session Resumption):** Configurar el ESP32 y el broker para reutilizar las credenciales criptográficas de la sesión anterior en reconexiones breves. Esto evita ejecutar el handshake asimétrico completo desde cero y reduce la latencia a pocos milisegundos.
- **Establecimiento de FQDN locales mediante wrappers:** Adoptar implementaciones orientadas a validar de forma estricta los certificados criptográficos sobre nombres de host locales (como `mqtt.local`) empleando resolutores personalizados por IP, evitando saltarse la seguridad de la cadena de confianza (evitar el uso de `setInsecure()`).
- **Autenticación mutua (mTLS):** En implementaciones industriales con alta exposición, escalar el canal TLS hacia una autenticación bidireccional, requiriendo que los dispositivos IoT también presenten un certificado firmado al broker para consolidar un canal libre de suplantaciones.